Week 8 Milestone Report: Proposal and Design Defense

Course:	CS3320 Database Systems, Instructor Dr. Chibuzo Ukegbu
Student:	Shritej Gunda (individual project)
Project:	Factory Sherlock: Manufacturing Simulation, Quality Control, and Defect Investigation on MySQL
Date:	2026-11-03
Repository:	https://github.com/shritej6/factory-sherlock
Commit or tag: `milestone-1`	tag (Week 8 submission)

1. Completion status, incomplete items, and blockers

Completed for this milestone:

- Proposal document (docs/milestones/week08-proposal.md, submitted as PDF) covering the problem, users, workflows, MVP boundary, stretch goals, draft ERD, normalized tables, relationships, assumptions, constraints, business rules, technology stack, query ideas, security plan, testing plan, and AI plan.
- Draft ERD with 17 tables and three many-to-many junction tables (batch_materials, inspection_defects, investigation_batches).
- Repository skeleton with backend, frontend, database, docs, and scripts directories.
- Starting-point reflection (section 8 of the proposal; written personally by the student).

Incomplete or in progress:

- The frontend is implemented (React 19, TypeScript, Vite 7, Tailwind 4, React Router 7, TanStack Query 5, Recharts 3) with screens for Login, Overview, Machines, Orders, Batches, Inspections, Traceability, Investigations list, Investigation detail (case board with evidence cards and show-the-SQL), Analytics (charts plus SQL Explorer), and Simulation. Nine screenshots were captured from the running application on 2026-11-03 (docs/media/screenshots); no video has been recorded.
- Automated tests and the GitHub Actions workflows exist: `uv run pytest` last reported 100 passed (95 percent coverage of the backend `app` package) and `pnpm test` 39 passed in 13 files; `.github/workflows/` holds `backend-ci.yml`, `frontend-ci.yml`, and `docs-lint.yml`. All three workflows passed on the first push to GitHub on 2026-11-04 (Backend CI run 41779775539), and the test matrix (docs/testing/test-matrix.md) records which scenarios are only partly covered.
- The Docker path (docker-compose.yml with scripts/setup/init_databases.sh mounted as the init script) has not been executed, because Docker is not installed on the development machine; the local MySQL path (scripts/development/local_mysql.sh) is the one that has been run.
- The supporting documentation sets (docs/database, docs/simulation, docs/investigation, docs/requirements, docs/development, docs/testing) exist; the walkthrough video is the remaining media item.

Blockers: none.

2. Required technical artifacts and repository commit or tag

Artifact	Location
Proposal PDF	docs/milestones/week08-proposal.pdf (also submitted to Canvas)
Draft ERD (Mermaid source)	docs/milestones/week08-proposal.md, section 4; docs/database/database-design.md
Schema draft (ORM models)	backend/app/models/*.py
Query ideas	database/queries/*.sql and database/queries/investigation/*.sql
Repository commit or tag	tag `milestone-1` on https://github.com/shritej6/factory-sherlock

3. Decisions, alternatives, assumptions, constraints, and rationale

Decision	Alternatives considered	Rationale
Modular monolith: one FastAPI service with api, services, domain, repositories, simulation, and investigation packages	Separate services for simulation and investigation; a single flat module	One process is reproducible on a laptop and keeps transaction boundaries in one place; packages still separate HTTP concerns from business rules so rules can be unit tested without a database
Request-scoped transaction: one SQLAlchemy session per request, committed on success and rolled back on any exception	Explicit commit in each service; autocommit	A handler that fails halfway through a multi-table write leaves no partial rows; services never manage transactions themselves
Analytical queries kept as raw SQL files loaded by name	ORM query builder for everything	The SQL the instructor reads is the SQL that runs, and the investigation case board shows the same text to users
Units stored as rows with serial numbers	Store only counts per batch	Backward traceability from one serial number requires a unit row; inspections attach to units
Stored pass or fail result on each inspection	Recompute from specification at read time	Historical results stay stable if a tolerance is later changed
ENUM columns for statuses	Lookup tables; free text	The database rejects unknown values even when rows are inserted outside the application; the value sets are small and fixed

Assumptions: every unit is inspected on every characteristic of its product; a batch runs on one machine for one order; timestamps are plant local time without time zone. Constraints: MySQL is required by the course; everything must run locally and free; the project is solo, so scope is limited to the MVP described in the proposal with stretch goals listed separately.

4. AI help: what, why, accepted, revised, rejected, and verification

Tool: Claude Code (Anthropic). Codex and Gemini CLI are planned as secondary tools for independent review. The student reviews and verifies every component before it is kept.

Stage	Task	Recommendation	Decision	Verification
Design	Backend structure	Proposed the modular monolith layering (api, services, domain, repositories, simulation, investigation)	Accepted	Reviewed against the course requirement that business rules be enforced server-side; the layering is implemented and the backend runs against MySQL 8.4
Design	Transaction handling	Proposed the request-scoped transaction pattern (session per request, commit on success, rollback on exception)	Accepted	Confirmed by running the code against MySQL 8.4: a failing multi-table write leaves no partial rows

Lesson recorded: an AI proposal for structure is useful as a starting point, but each pattern is kept only after it is implemented and exercised against the real database.

5. Lessons learned at this milestone

- Writing the business questions before the SQL made the query list concrete and tied each query to a screen or investigation rule.
- Deciding delete behavior (RESTRICT, CASCADE, SET NULL) per foreign key at design time exposed which tables are history and which are owned child rows.
- Keeping the MVP to five modules plus the dashboard, with stretch goals in a separate list, made the scope defensible for a solo student.

6. Support or skill development needed for the next stage

[State any instructor support or skill development you need for Week 9, or write "None at this time".]

7. Next-step plan (Week 9)

1. Finish the README setup draft and the project structure overview (docs/development).
2. Produce MySQL connection evidence: database created by scripts/setup/init_databases.sh, Alembic migration applied, /health endpoint responding.
3. Complete the seed plan: reference data in database/seeds/01_reference_data.sql, demo accounts from app.seed, demo scenario in database/seeds/demo_scenario.json.
4. Record every AI interaction that affects a decision in docs/ai_evidence.

8. Personal reflection (Shritej Gunda)

[Write 150 to 250 words in your own voice. Replace each prompt.]

[What did you do yourself at this milestone, and which parts did you check most carefully?]

[What was harder than expected while writing the proposal and the ERD?]

[How did AI help or get in the way at this stage, with one concrete example?]

[What will you do differently in Week 9?]

9. Personal-work summary

[List the artifacts you personally produced or verified for this milestone, one line each, with the file path or screen name.]